Using Machine Learning to Predict Optimal Erasure Coding Policies for Object Storage System in OpenStack Swift

Amio Malakar Ankon[a], Boloy Chaki[a], Debashish Sarker[a], Mashrur Shakhawat Sayan[a] and Jannatun Noor[b]

[a]Department of Computer Science and Engineering, BRAC University, Dhaka, Bangladesh
[b]Department of Computer Science and Engineering, United International University, Dhaka, Bangladesh

ARTICLE INFO

Keywords:
Cloud Computing
Object Storage System
Machine Learning
Erasure Coding
OpenStack Swift

ABSTRACT

Erasure coding (EC) reduces storage overhead and improves fault tolerance in object storage systems, but selecting an appropriate EC policy often involves trade-offs among latency, recovery behavior, storage efficiency, etc. Service providers like Ceph or OpenStack Swift use benchmarking or administrative configuration for selecting EC policies without considering exact workload contexts. To address this problem, this paper showcases a data-driven performance modeling framework using machine learning to suggest optimal EC policy. A structured dataset- ABDS-30k is constructed in a controlled execution of varying workload conditions on a Swift All-In-One testbed under different erasure coding policies with the addition of failure injections. It collects several empirically observed performance metrics such as read and write latency, tail latency, success rate, and reconstruction time in case of disk failures. We adopt two distinct machine learning paradigms- a regression-based approach of performance modeling and a classification-based approach for direct data-driven policy recommendation. Experimental results show that the regression models have moderate prediction errors for latency and recovery metrics because of the inherently noisy and heavy-tailed nature of the system, but they remain effective in optimal policy recommendation by exhibiting top-1 accuracy with 48.16% in XGBoost Regression and a top-3 accuracy 100% in Random Forest Regression model. Classification approach shows comparatively weaker metrics, indicating that it is not optimal for data-driven policy recommendation. Overall, this shows the feasibility of using ML-based performance modeling, which can be later used as a foundation for future control-plane automations.

1. Introduction

In the current data-driven world, most business models rely on cloud storage, increasing the need for cost optimization. Reduction of latency, the delay in data transfer, is essential for enhanced user experience. In cloud architecture, to meet these challenges, object storage systems are becoming famous day by day, which is a type of data storage architecture used to manage and store unstructured data while ensuring scalability. This considers units of data as objects that contain raw unstructured data in the form of images, videos, etc, and metadata, which is information about the object and a unique global identifier. In today's day and age, all intelligent devices and machines create insane amounts of data, such as cookies, data packets, information, etc. These data are constantly driving the servers towards the edge of their capacity, often overwhelming and sometimes even hanging up whole complete systems. These create bottlenecks in the system that normally slow down the complete system and in severe cases, may even cause the loss of important data connections and packets. According to Lui et al. [1], these bottlenecks bring forth the necessity of reducing storage overhead and efficiency- two key aspects essential to improving responsiveness and smooth data flow in a system. OpenStack Swift, an object storage system, uses erasure coding, a novel method to store data, which is an alternative to replication and has lower storage overhead and fault tolerance. In OpenStack Swift, erasure coding policies are administratively assigned and implemented when objects are uploaded to the system. But, the selection policy of erasure coding policies are not simple and does not strictly depend on partitions, with various metrics like read and write latency, tail latency, success rates, reconstruction time with many more factors which depend on workload and system configuration. In this paper, we create a system based on machine learning that models object upload, download, and reconstruction behavior under varying workloads and failure conditions, and recommends an optimal erasure coding policy using a weighted, performance-aware scoring strategy.

In current IoT and cloud computing, erasure coding has become an indispensable part of distributed storage systems because of its effectiveness in data distribution and durability with robust data [2]. Erasure coding stores data by encoding it and partitioning it into k+m cells, where k is the number of data cells, and m represents the number of parity cells [3]. Although this achieves better space efficiency than replication, performance degradation still happens when the volumes of k and m increase [3]. So, erasure coding by itself is not always perfect, and the selection of erasure coding can not always be determined by the combination of k & m. In

amio.malakar.ankon@g.bracu.ac.bd (A.M. Ankon); boloy.chaki@g.bracu.ac.bd (B. Chaki); debashish.sarker@g.bracu.ac.bd (D. Sarker); mashrur.shakhawat.sayan@g.bracu.ac.bd (M.S. Sayan); jannatun@cse.uiu.ac.bd (J. Noor)

ORCID(s): 0009-0002-1245-1039 (A.M. Ankon); 0009-0002-0840-9241 (B. Chaki); 0009-0005-4096-0510 (D. Sarker); 0009-0003-2556-075X (M.S. Sayan)

our work, we address this gap by evaluating EC policies in an OpenStack Swift testbed by executing numerous workload scenarios and building a machine learning model that helps to determine the optimal EC policy.

Most of the existing research and literature have evaluated EC schemes only within a few controlled workload variations and work on improving EC itself based on its fault tolerance, robustness tolerance etc metrics. Some papers, such as MSR Cambridge, Harvard NFS provide traces for workload realism for EC evaluation [4], but they do not provide an explicit overview of erasure coding policy per workload. So we created our own dataset by using OpenStack Swift and testing EC coding policies in various workloads, and creating an ML-based system that predicts optimal policy. EC in OpenStack Swift uses actual fragmentation, encoding, and decoding, so our dataset contains real client I/O and backend work metrics.

In the context of erasure-coded storage systems, relationships between configuration parameters, workloads, failure conditions, and performance metrics are highly complex. Traditional static policy selections cannot fully ensure the optimal policy selection due to the intricate nature of the dependencies. By using ML techniques on performance data, we can create a predictive policy recommendation system from system state and workload features. This can allow administrators to find the optimal policies for their specific efficiency, latency, and robustness needs.

In the current world of IoT, it is estimated that 2 billion terabytes of data will be stored in cloud storage by 2025, and by 2029, the cloud market is projected to be worth 376.36 billion USD [5]. This showcases the significance of storage policies in object storage systems to keep up with the market. Additionally, Gartner forecasts that worldwide public cloud end-user spending will reach $723.4 billion in 2025 [6]. This shows the rapid growth of cloud adoption and the increasing economic importance of efficient storage management. In systems like OpenStack Swift, with each new workload, the configuration has either to be tested empirically for policy selection or chosen based purely on partition numbers- which is not always optimal. The main motivation behind this project is to create a framework that suggests an optimal EC policy given a specific workload scenario based on authentic data. Using a machine learning framework can give a proper assessment of optimal policy of a system within milliseconds where real system simulation and checking often cost time and money.

In the industry, service providers like Ceph [7] do benchmarking based on throughput/latency without considering workload diversity. OpenStack Swift [8] itself allocates erasure coding policies administratively by creating a ring and disks in a static, manual method without the scope considering any workload context beforehand.

In addition to the EC configuration, an object storage system has numerous factors that determine the upload, download latencies, and reconstruction time of objects stored in it. While various models for enhancing storage performances exist, there is a lack of authentic data-driven models that can evaluate all relevant metrics related to EC system and recommend optimal policies based on the specific workload context.
So, this makes it impossible to systematically recommend an optimal policy without empirical testing for each new workload. Therefore, we address this core problem by suggesting a machine learning based framework to optimally select erasure coding policies for varying workloads and mitigating the limitations in static configurations.
The primary objective of this study is to develop and evaluate a machine learning based framework that can suggest optimal erasure coding policies in OpenStack Swift. To achieve this objective:

- A controlled OpenStack Swift All-In-One (SAIO) cluster is created and a workload generator is developed to generate benchmark workloads across the configured erasure coding policies.
- The workload generator is executed across the configured EC policies and under induced failure conditions to create a structured dataset suitable for machine learning.
- Using the created dataset, separate classification and regression models are trained and validated for suggesting optimal EC policy.

Thus, we evaluate the overall feasibility of the ML-based approach of finding an optimal EC policy for a workload context. In this research we aim to propose a machine learning framework that can suggest the optimal erasure coding policy based on workload context in object storage systems. For this we simulate authentic workload contexts in a testbed and create a dataset, then train and evaluate the machine learning models on suggesting the optimal EC policy:

1. OpenStack Swift is used as a testbed to simulate and generate the workload contexts, and the dataset-ABDS-30k is generated based on that.
2. The dataset is used as an input for the machine learning framework, where two separate paradigms are used based on classification and regression.
3. Both the regression and classification-based models are trained on the dataset and predict the optimal policy for a specific workload context based on a specific score, which indicates the upload, download latency, and object reconstruction time.
4. The best optimal policy is recommended to the user, which may further change in the future based on the then-current usage of the user and the workload context.

Our experimental results demonstrate that no single EC policy is suitable or optimal across all workload and system conditions. The performance and cost-outcomes depend on the workload characteristics. ML can be implemented to ensure that it effectively models these complex relationships to improve storage efficiency and select an optimal EC policy.

2. Background

2.1. Object Storage System

Many cloud object storage systems have become fundamental infrastructure for modern analytical database systems [9]. An object storage system organizes data into immutable objects stored within a container called buckets. Each object has its own unique identifier and metadata. Object storage systems use a distributed architecture with no central point of control, providing greater scalability, redundancy, and permanence. Object storage is ideal for cost-effective and scale-out storage. There are many object storage systems like AWS S3, Google Cloud Storage, OpenStack Swift, etc. OpenStack Swift provides a fully RESTful API, enabling programmatic access to scalable and redundant object storage.

2.2. Erasure Coding

Erasure coding (EC) is a method of data protection used in a distributed storage system to minimize storage overhead [10]. Unlike the replication method, which stores a full copy of data in multiple storage locations, erasure coding divides data into fragments, adds redundant parity fragments, and distributes them across multiple nodes. Even if a drive fails or data becomes corrupted, the data can be reconstructed from segments that are stored on other drives. Here, the original data is split into k data fragments and generates m parity fragments. So, the total number of fragments is n=k+m. Erasure coding saves storage space significantly and provides higher fault tolerance by allowing data recovery from multiple concurrent failures. Even with many advantages, this has a computational overhead for encoding and decoding with reconstruction latency.

3. Related Work

This section reviews prior work related to erasure coding, object storage system optimization, and machine learning techniques used in storage management. We group the studies of researchers by topic and discuss what they did.

3.1. Erasure Coding:

Noor et al. [2] benchmark multiple erasure coding schemes to evaluate time efficiency and fault tolerance in the OpenStack Swift object storage system, specifically on the three Solomon-Reed schemes. They highlight the core performance bottleneck: as the number of fragments and parity increases, upload, download, and deletion times increase. However, this study primarily investigates the efficiency tradeoff between I/O performance without extensive testing on real-world dynamic cloud situations. Their methodology revolves around 2 testbeds, both local and remote, along with the SimEDC simulator to establish a benchmark (COCO-17) for a custom dataset (MCSD-100). In key findings in this study is that if the number of fragments and parity data is increased, it concurrently leads to longer upload, download, and deletion time being the downside.

To address EC overhead in system operations, Pei et al. [11] propose Group U, which is a grouped and pipeline scheme that is designed to update in erasure-coded distributed systems. To counter high I/O overhead and low efficiency, this research proposes a radical new idea of using a hybrid three-layer framework that would support both single and multiple updates. Their work on RDFS shows time reduction by 21%-69% and overhead reduction by 22%-46% compared to PUM and PDP-P. It shows that the update-path optimizations at the storage-system level can significantly reduce time and overhead in erasure-coded storage.

A novel approach is proposed by Hu et al. [12], where they developed high-performance erasure coding libraries by leveraging existing optimized ML libraries rather than building custom implementations. They focus on EC library design and acceleration for fault-tolerant storage rather than evaluating a specific object storage system's end-to-end behavior. They developed EC libraries using Apache TVM. Traditional EC libraries adapt to heterogeneous platforms, including GPUs, CPUs, and accelerators, and demand a high level of expertise in both coding theory and low-level hardware optimization. Their study highlights the bridging of optimized compute libraries with storage-system internals.

For retrieval-side optimization, Cui and Wang [13] suggest using EC-KAD, a scheme that uses erasure coding integration with Kademlia DHT protocol for optimization of object storage. The working order of this scheme is to use a Reed-Solomon code that is used for fault tolerance, and also uses the Kademlia protocol for efficiently locating data blocks.

Finally, a study by Xu et al. [14] evaluate incremental encoding to optimize new data encoding in erasure-coded storage systems using a decentralized framework. Evaluations show a better trade-off compared to other models with 44-48% less network traffic, even with the same I/O performance as replication encoding, and still retaining superior read/write performance than strip encoding.

3.2. Object Storage System:

In metadata management for cloud-scale object storage, Chen et al. [15] propose a hybrid model that uses a mix of SSD and HDDs to access CEPH and OASIS models. Here they are using SSD for bigger data and frequently accessed data, and HDD for smaller ones. They use the RADOS service through the CLUSTER algorithm to complete this.

For efficient object access and retrieval, Durner et al. [9] give us a brief description of how cloud object storage can be used efficiently and cost-effectively by presenting a cloud object storage download manager with a brief analysis of cost, latency, throughput, and performance compared to state-of-the-art database systems. They discuss challenges

such as latency under concurrent requests, CPU overhead under large analytic workloads, and multi-cloud complexity. They are proposing an approach with AnyBlob and integration with Umbra DBMS, where they achieved maximum performance using 0.7 times the CPU resources and 1.5 times the performance under a fixed CPU budget. Additionally, they reduced CPU usage, showing optimization of resource utilization and reducing computational overhead.

Another study by Bucur et al. [16] provides a review of object storage in single-cloud and multi-cloud environments, which highlights the lack of unified storage solutions across major providers. They analyzed individual cloud providers compare their performance, architectural approaches, and storage classes. They discussed challenges, like security, multiple SLAs, and implementation. To address these issues, they propose autonomic SLA management systems, vendor-agnostic APIs, and secure data distribution models.

In multi-cloud, Matt et al. [17] use erasure coding to split objects into chunks and distribute them across different cloud services. Then they evaluate placement based on pricing models, availability, access patterns, and latency. They address the challenges of efficiently storing data across multiple cloud storage providers. They presented a heuristic optimization approach that optimizes storage placement, cost, and latency efficiently while considering user-defined quality of service (QoS) requirements.

A study by Factor et al. [18] described an object storage system and T10 OSD protocol (version 1) as a self-managed storage access via cryptographically enforced capabilities. They showed throughput results for object storage controllers, while noting adoption barriers due to the shift from block-based interfaces.

Again, a study by Su et al. [19] propose an analytical performance model to predict response latency for cloud object storage systems. They achieved 4.44% prediction error in their model, but in their variety of scenarios, they reduced the prediction errors by up to 73% compared to baseline models. From their research, they validated on OpenStack Swift, enabling efficient capacity planning and SLA compliance.

A study by Akbar et al. [20] addresses the challenges of data deduplication in a cloud storage system. They propose a hybrid ML-based approach to achieve efficient deduplication and increase security by securing data through encryption, which conflicts with deduplication in cloud environments.

This study by Levitin et al. [21] proposes a probabilistic model that optimizes the uploading and downloading pace distribution between non-identical storage units, where both systems are used to maximize MSP. The authors provide a framework for evaluating MSPs under system demand.

Finally, Saadoon et al. [22] provide a broad survey of fault tolerance mechanisms and trade-offs in large-scale storage and processing systems.

3.3. Machine Learning:

A study by Hu et al. [12] used ML libraries to accelerate EC. They mapped erasure coding to general matrix multiplication (GEMM), showing that bitmatrix coding is structurally identical to GEMM. Relying on TVM's auto-tuning and hardware optimization, they defined the erasure coding within 40 lines of codes and they compared TVM-EC against state-of-the-art libraries on a CPU platform using the Reed-Solomon algorithm. With that, they achieved up to 1.75x higher throughput than custom libraries. Also, they pointed out that ML libraries may perform badly with hardware with few parallel computing resources. Also, they face integration complexity because ML libraries expose a high-level API, making integration with a low-level storage system non-trivial.

Mondal et al. [23] focus on RSoS, an object-based schema-oriented data storage system, which handles various types of data across multiple database backends. They evaluated a classification framework that predicts the appropriate storage location for incoming data, such as healthcare scenarios, where manually configuring storage metadata is error-prone. They use Ganglia to collect system metrics each time a query hits RSoS. For the feature selection, they compared three supervised algorithms and tested three different classifiers. They achieved 100% accuracy, precision, and recall in predicting storage location for three data types using LII21 + K-NN. The system correctly separated data types in the databases.

A research paper by Noel et al. [24] developed an Adaptive Resource Management (ARM), which uses a Reinforcement Learning (RL)-based approach. It enables the self-managing cloud storage system by automatically balancing load and migrating data in response to performance hotspots. They implemented and evaluated on Ceph, an open-source distributed object storage system. They collected system metrics, used a stochastic policy gradient RL algorithm to decide which one to take action in the correct time, and executed the action by adjusting Ceph's OSD weights. The performance monitor fetches the system data from the Ceph OSDs at 30 seconds. They achieved up to 50% faster read response and 33% faster write response time compared with the default Ceph.

A study by Syed and Albalawi [25] tries to use machine learning to efficiently allocate resources for cloud computing services. They use prior history and cloud scalability to train the model to give them a machine-learning model that would optimize cloud resource management, using predictive models, optimization algorithms, and real-time adaptation.

A study by Kamble [26] focuses on resource allocation using CNN, LSTM, and Transformer deep learning algorithms that help the Cloud Service Providers (CSP) to improve cost-effectiveness, performance, and resource utilization. The authors address practical issues like cost, performance, and operational efficiency while leveraging real-world datasets (Google Cluster Data) for credible analysis.

Table 1
Table of Key Findings

Reference	Used Algorithms / Architecture	Sector of optimization	EC Used?
Noor et al., 2025 [2]	Reed–Solomon (RS) codes (RS(5+3), RS(7+5), RS(10+4)); SimEDC simulator; OpenStack Swift.	Time efficiency & fault tolerance benchmarking for I/O performance in OSS.	Yes
Pei et al., 2016 [11]	Group-U: a three-layer framework with workload-aware grouping and pipelined transmission.	Update efficiency for in-place updates in erasure-coded storage.	Yes
Levitin et al., 2023 [21]	Probabilistic model using Proportional-Hazards Model (PHM) / Accelerated Failure-Time Model (AFTM).	Mission reliability and pace-dependent failure in production-storage systems.	No
Cui and Wang, 2025 [13]	EC-Kad: RS erasure codes with Cauchy matrix + Kademlia DHT protocol.	Fault tolerance & retrieval efficiency in cloud storage.	Yes
Xu et al., 2018 [14]	Incremental encoding: a decentralized, pipelined encoding framework for linear codes.	Encoding efficiency and network traffic for writes in geo-distributed storage.	Yes
Kamble et al., 2024 [26]	LSTM, CNN, BERT.	Resource utilization, cost-effectiveness, and performance.	No
Saadoon et al., 2021 [22]	Survey/review: classifies fault types (permanent/transient) and tolerance approaches (detection, recovery).	Fault tolerance in big data storage and processing systems.	Discussed
Muntala and Karri, 2023 [27]	Conceptual framework using Oracle Cloud Infrastructure (OCI) Data Science and Autonomous Database.	ML lifecycle management for predictive analytics in ERP.	No
Patil and Sharma, 2020 [28]	Machine learning for feature extraction and optimization; implemented in Python with TensorFlow.	Memory management for cloud/mobile environments.	No
Su et al., 2017 [19]	Queuing-theory performance model.	Tail latency percentile prediction.	No
Matt et al., 2017 [17]	Heuristic placement optimization of cost/latency-aware policies.	Multi-cloud placement to reduce cost & latency under QoS constraints.	Yes
Mondal et al., 2021 [23]	Supervised classification for storage placement.	Storage placement recommendation.	No
Durner et al., 2023 [9]	AnyBlob, object scheduler, and object storage architecture.	Storage cost optimization and high-performance data retrieval.	No
Su et al., 2021 [29]	ARIMA, LSTM, DQN, PSO, linear regression.	Private data sharing in edge-cloud collaboration.	Yes
Syed and Albalawi, 2024 [25]	Predictive ML and optimization framing for resource allocation.	Resource management optimization.	No
Chen et al., 2024 [15]	Hybrid metadata system for hot/cold metadata handling.	Metadata maintenance.	No
Noel et al., 2019 [24]	RL-based adaptive resource management.	Response latency requirement.	No
Akbar et al., 2023 [20]	TTD+DPC chunking, multi-level hashing.	Dedup ratio + throughput + CPU overhead reduction.	No
Bucur et al., 2018 [16]	Review of single-cloud and multi-cloud object storage systems.	Multi-cloud object storage architecture.	No
Factor et al., 2005 [18]	T10 OSD protocol framing.	Object storage architecture foundation.	No
Hu et al., 2024 [12]	TVM-based EC library mapping.	Reduced developer complexity and faster EC implementation.	Yes

A paper by Muntala and Karri [27] shows a conceptual framework to manage end-to-end machine learning lifecycle with Oracle Cloud Infrastructure. Its main focus is on ERP-related applications where its core attributes are to build an architectural guide to use in the integration of OCI data with Oracle Fusion ERP data to build an advanced predictive analytics to be used for expense forecasting.

A research paper by Patil and Sharma [28] proposes an ML-based algorithm for resource allocation in mobile and cloud computing environments. The novel approach used in this study was to use feature extraction and optimization algorithms for eliminating unused cache files to reduce data blocks and device storage overhead.

In distributed object storage, Wang et al. [30] propose DanceNN to reduce memory bottlenecks and improve metadata handling, which can improve retrieval performance and overall scalability. They also address the demands for computational and storage resources, because cloud computing and machine learning algorithms are evolving. They apply ML for performance prediction in storage systems, using a Random Forest model to analyze the performance patterns and bottlenecks.

4. Dataset

In this chapter, we discuss the design, generation method, and characteristics of the dataset developed for this thesis-ABDS-30k [31]. For the construction of this dataset, thirty thousand workload variations are executed on OpenStack Swift as a storage system under multiple administratively assigned erasure coding policies and controlled failure injections. So, each of the samples in the dataset represents an independent and unique experiment of a workload.

4.1. Motivation and Objectives

In the current world, most object storage systems, such as OpenStack Swift, offer multiple erasure coding policies to store objects, which vary in storage efficiency, latency, and fault tolerance. But, the selection of an optimal erasure coding (EC) policy is complex and includes several characteristics based on workload, system, and recovery behaviour in case of failures. Existing literature in this sector mostly focuses on only a few workloads and some hand-picked scenarios, such as comparison and benchmarking between EC schemes and replication [2]. Some of the works focus on using public traces such as MSR Cambridge, Harvard NFS, as seen in the study done by Chan et al. [4], but those are not explicitly documented outcomes based on variable workloads per EC policy. Hence, we created ABDS-30k for capturing the real performance of an object storage system in varying workloads and configurations with failure injections to use it for machine learning and recommend optimal policies based on the target features.

4.2. System Context and Experimental Assumptions

We created a realistic, small-scale object storage cluster in OpenStack Swift running in Ubuntu 18.04.6 as a virtual machine. Swift Train version was used for creating a single-node Swift cluster. We also used 14 XFS-formatted virtual disks for the storage backend. For erasure coding, we used 10 erasure coding profiles, which are EC-2-2, EC-3-2, EC-4-2, EC-4-3, EC-5-3, EC-6-3, EC-7-3, EC-8-3, EC-8-4, and EC-10-4.

4.3. Data Collection Architecture and Workflow

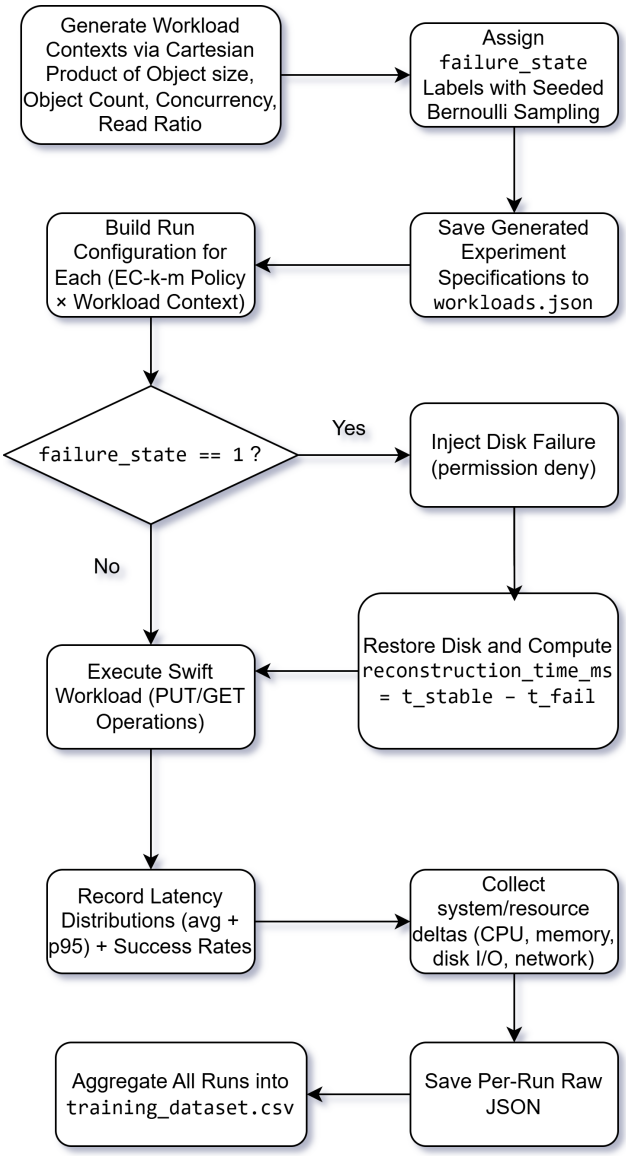

Figure 1: Dataset Creation

Figure 1 shows our dataset creation process. The data collection process is designed to benchmark OpenStack Swift's different EC policies, workloads, and failure scenarios to generate a structured dataset. Here, architecture and workflow include the following components:

- Workload configurations contain file or object size, number of objects, concurrency level, and read/write ratio.

- Uploaded/downloaded data is generated by multiplying a randomized byte to create larger files to simulate real-world data.
- These workloads execute object upload and download operations using OpenStack Swift APIs and measure latency.
- Failure injection, which simulates disk failure by modifying disk permissions to emulate degraded storage conditions.

All workflows are executed using the Ubuntu terminal and Python automation tools.

4.4. Workload Design and Scenario Space

The dataset was created using a virtual machine with 8 GB of base memory and 12 CPU cores. The virtual machine was running on a computer with an Intel Core i5-12500H and 16 GB of RAM. The dataset was generated on a virtual machine with Ubuntu 18.04.6. Also, OpenStack Swift 2.23.3 (Train) [32] was used to create a storage cluster with erasure-coding capabilities. In our generated 30,000 labeled samples, the dataset includes six primary input features describing the workload and system. Each workload takes EC policy identifier, file size, number of objects, concurrency level, read ratio, and failure state. Each workload is a combination of different parameters. Here we set our object size or file size from 16 KB to 1 MB, number of objects is between 1 and 10. Concurrency level is the number of parallel client threads, and read ratio is the fraction of objects that are read after upload.

4.5. Failure Injection and Recovery Modeling

Failure injection is modeled by temporarily simulating disk unavailability at the storage layer during workload execution. OpenStack Swift system detects failure when the selected storage device is made inaccessible using permission-based access denial. During failure, Swift object servers experience real I/O failure. It triggers EC fragment construction during reading. Swift avoids destructive disk removal or system crashes. In the workload, 40% of runs are executed under simulated failure conditions. So, each workload-policy pair is tagged with a failure state label or failure_state=1. This temporarily makes one storage disk unavailable, and the system is forced to operate in degraded mode. This triggers erasure coding recovery behavior. A recovery-oriented metric, reconstruction_time_ms, captures the time to access data under failure conditions.

4.6. Feature Engineering and Target Variables

Each line of the dataset represents an experiment run. The ML works here as a multi-target performance predictor with one regression model implemented per target metric. The features used as input features include system state, workload context, policy configuration, etc. The outputs are then measured in metrics that are also visible to the user.

Targets (regression outputs):

- avg_upload_ms
- p95_upload_ms
- avg_download_ms
- p95_download_ms
- reconstruction_time_ms

The recommendation steps a wide variety of EC policies (e.g, 4-2, 2-2, 6-3, etc) to find the most efficient one. These policies are written as EC-k-m, where k = data fragments and m= parity fragments. The policy storage properties are derived from these equations:

- **Storage overhead factor:** storage_overhead = $\frac{k+m}{k}$
- **Storage efficiency:** efficiency = $\frac{k}{k+m}$

These are used in the resulting score to determine the efficient policy.

4.7. Dataset Construction Workflow

For dataset creation we created a workload generator to generate 3,000 workload combinations while assigning failure state label to 40% of them and save them in a json file. After that, we run the experiment and collect raw results in one json file per run. For failure states, we inject disk failure (permission deny). After all 30,000 runs are complete, we aggregate these results into a final csv file.

4.7.1. Dataset Structure and Column Groups

The column metrics of the dataset can be classified into 5 major categories: policy parameters, workload characteristics, failure & degradation indicators, system resource metrics, and performance & I/O metrics. Each category gives a broader, detailed picture of what's happening in that respective sector. Policy Parameters These columns explain the policy parameters in the EC configuration:

- **policy_name:** Policy identifier (e.g., EC-6-3), representing a specific (k, m) combination.
- **Ec_k**: number of fragments (k) in which the data is divided.
- **Ec_m**: number parity (m) bit for reconstruction and redundancy.
- **Storage_overhead**: calculated as (k+m) /k for redundancy cost
- **Efficiency**: calculated as k/(k+m)

In the experiment, these columns explain that higher redundancy may provide more durability, but also comes at increased encoding/decoding cost and reconstruction complexity.

Workload Characteristics:

These columns are to measure the workload applied to OpenStack Swift during each experiment:

- **workload_id:** Identifier of the workload scenario/pattern.
- **File_size:** Object size
- **Num_object:** Number of objects
- **Concurrency:** Number of parallel client operations.
- **Read_ratio:** What fraction of operations were read (0.0 = write-only, 1.0 = read-only).
- **total_mb, read_mb, write_mb:** Total data breakdown of each operation
- **Io_intensity:** Indicator of workload pressure/strength.

Failure and Degradation Indicators:
These columns explain failure conditions:

- **failure_state:** Explains if a failure state happened
- **failed_disks:** Number of disk failures during the run.
- **Failed_nodes:** Number of node failures
- **time_since_last_failure_s:** Time since last failure event.

System Resource Metrics:
These columns describe the system state before and during each workload execution:

- **data_disks_used_pct_before/after/delta:** Disk space utilization before and after the run.
- **cpu_util_pct_during_run:** CPU usage during workload execution.
- **mem_available_kb_before:** Available system memory before the run.
- **load1_before:** System load average prior to execution.
- **runnable_procs:** Number of runnable processes in the system.
- **dirty_kb, writeback_kb:** Indicators of memory write pressure.

Performance, Network, and I/O Metrics:
The rest of these features explain the overall performance, network activity, and I/O pressure on the system:

- **net_rx_bytes_delta, net_tx_bytes_delta:** Network receive/transmit traffic during the run.
- **disk_reads_delta_max, disk_writes_delta_max:** Peak disk I/O operations during the run.
- **disk_io_time_ms_delta_max, disk_weighted_io_time_delta_max:** Disk busy time and queue-weighted I/O time indicators.
- **disk_sectors_rw_delta_max:** Physical disk activity at the sector level.
- **avg_upload_ms, p95_upload_ms:** Average and tail (95th percentile) upload latency.
- **avg_download_ms, p95_download_ms:** Average and tail (95th percentile) download latency.
- **upload_success_rate, download_success_rate:** Operation reliability (success rate) under the given workload and failure condition.

4.8. Dataset Validation

To the best of our knowledge, no public dataset provides system metrics measurements for object storage systems in controlled workload and failure injection scenarios. Hence, we validate our dataset by mechanical correctness and invariant checks as per the official Object Storage System (OpenStack, Ceph) documentation and repeatability experiments in a separate testbed.

In ABDS-30k, the trends showcasing degraded read and reconstruction behaviour due to failure injection are supported by the behaviour reported in OpenStack Docs [33]. Also, the storage policy documentation from OpenStack states how different policies require different object rings, which validates our experimental configuration of policy-specific containers and rings [8]. Ceph's EC plugin benchmarks also support the variance in the values as per k/m and configuration noticed in our dataset [7].

For further validation and repeatability check, we reproduced a 120-sample version of the same dataset in a different testbed with 120 workload contexts and 3 policies and the noticed value trends were the same.

The following histogram (Figure 2) highlight the frequency distribution of reconstruction time for failure rows and how the outliers occur across the two datasets, where we notice a similar trend-

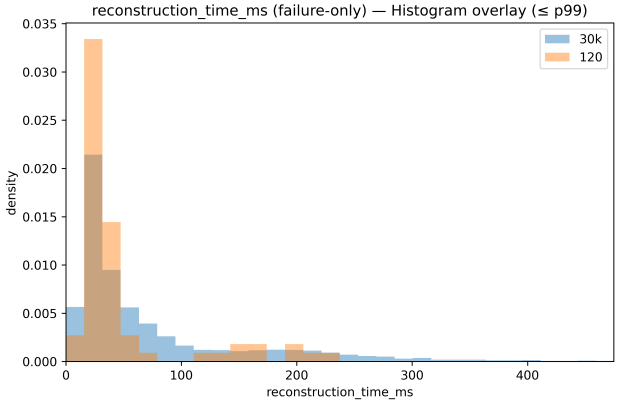

Figure 2: Histogram for frequency distribution of reconstruction time for failure rows

The following CDFs (Figure 3) highlight the failure cases that finish reconstruction under a specific amount of

time for both of the datasets, where we also notice identical trends in metric distributions-

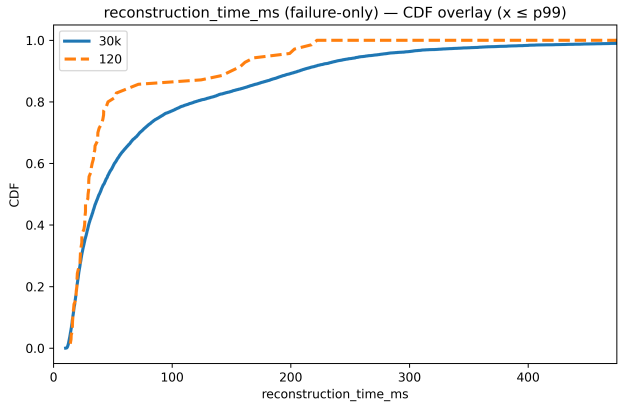

Figure 3: CDF for frequency distribution of reconstruction time for failure rows

The following graphs (Figure 4) showcase policy median±IQR (Interquartile Range), where IQR = Q3-Q1, is a measure of spread with Q1 is 25th percentile, and Q3 is 75th percentile - so the graphs show the variability by ignoring the outliers. Here we can see identical trends for both of the datasets, which validates the data variability-

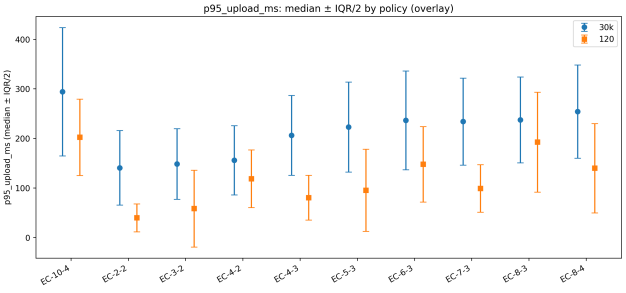

Figure 4: Comparison of median, p95 upload latency trends between the valida- tion dataset (120 samples) and the main dataset (30k samples)

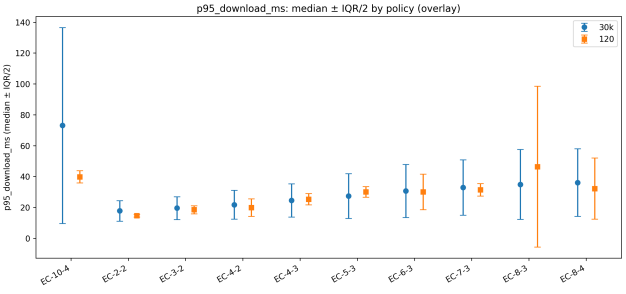

Figure 5: Comparison of median, IQR, and p95 download latency trends between the validation dataset (120 samples) and the main dataset (30k samples)

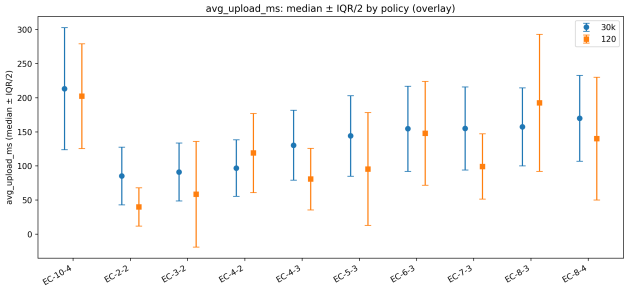

Figure 6: Comparison of median, avg upload latency trends between the valida- tion dataset (120 samples) and the main dataset (30k samples)

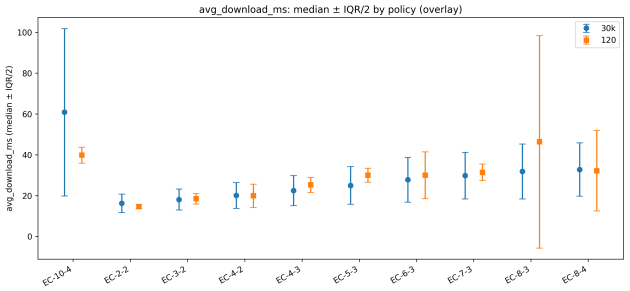

Figure 7: Comparison of median, IQR, and avg download latency trends between the validation dataset (120 samples) and the main dataset (30k samples)

5. Methodolgy

5.1. Design Process and Methodology Overview

This research investigates how erasure-coding (EC) policy selection can be automated using machine learning based on workload context, using OpenStack Swift as a testbed. To test the feasibility of EC policy recommendation, we generated a dataset of 30,000 execution records, covering 3,000 distinct workload contexts, where each context is executed under 10 candidate EC policies. The goal of the model is to recommend the EC policy expected to minimize the performance cost given a workload context. For this, we use two distinctive machine learning paradigm- regression and classification.

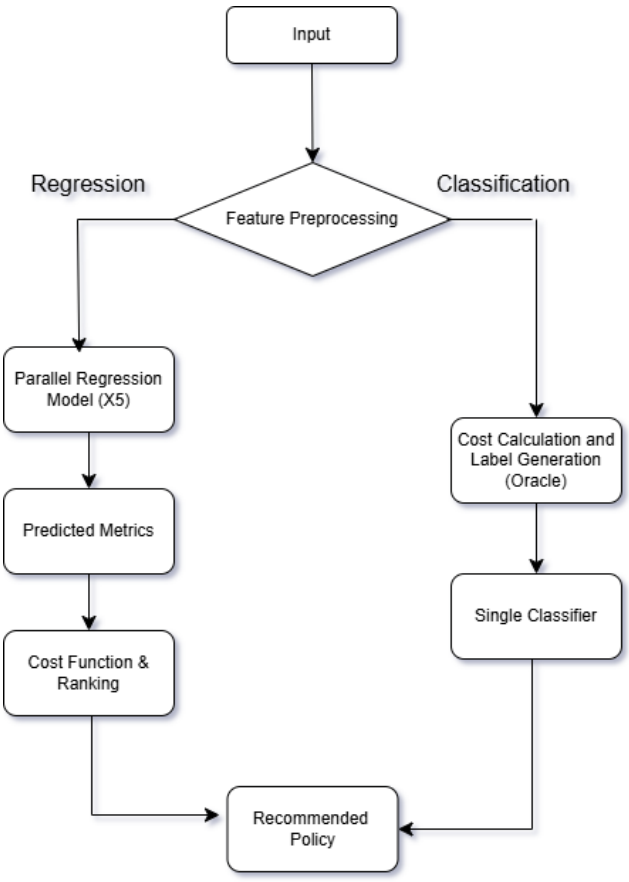

Figure 8: Proposed System Architecture

Figure 8 shows our full proposed system architecture. The first method uses regression, where three separate regression models- CatBoost, XGBoost, and RandomForest Regression are used for each of the target metrics, and the target features are predicted given a specific workload condition and EC configurations, and then scored based on a scoring function for recommending the best policy.
The second method formulates the problem as a classification task by computing a weighted score from measured latency and recovery metrics and by selecting the policy with the smallest score, a ground-truth label is created. Three classifiers- CatBoostClassifier, XGBoostClassifier, and Logistic Regression were used for this method.

5.2. Model Design Specification

The models use a feature set for learning that includes several types of metrics, such as:

- policy_name, failure_disk are categorical.
- file_size_mb, num_objects, concurrency, read_ratio, failure_state are basic workload parameters.
- total_mb, read_mb, write_mb, per_thread_mb, io_intensity, payload_bytes are derived workload parameters.
- data_disks_used_pct_before, mem_available_kb_before, load1_before, cpu_steal_pct_before, runnable_procs, dirty_kb, writeback_kb, swap_activity_delta are indicators of pre-run system state.
- disk_reads_delta_max, disk_writes_delta_max, disk_io_time_ms_delta_max, disk_weighted_io_time_ms_delta_max, disk_sectors_rw_delta_max are disk pressure indicators.
- failed_disks, failed_nodes, failure_disk, time_since_last_failure_s are used for failure situations.

The target features are:

- avg_upload_ms, p95_upload_ms which indicate upload latency.
- avg_download_ms, p95_download_ms for download latency.
- reconstruction_time_ms for failure recovery performance.

5.3. Regression-Based Design Specification

In this method, we train three regression-based machine learning models- CatBoost, XGBoost, and RandomForest Regressor on our dataset to predict a total of five target features. After training, the model is passed on the demo workload with the available EC profiles and is used to suggest an optimal EC policy based on the score that indicates weighted costs from the predicted values of the target metrics.

The whole workflow is done in four stages:

1. Dataset generation in a Swift All-In-One (SAIO) testbed by executing controlled workloads with injected failures under multiple erasure coding (EC) policies.
2. Selection of pre-run features that avoid leakage (i.e., only using measurements available before the workload executes).
3. Training one regression model per target metric: avg_upload_ms, avg_download_ms, p95_upload_ms, p95_download_ms, and reconstruction_time_ms.
4. Policy selection evaluation and recommendation based on the predicted performance for the given workload.

For policy recommendation, a normalized weighted score was defined with the values predicted of avg_upload_ms, p95_upload_ms, avg_download_ms, p95_download_ms, and reconstruction_time_ms with separate values of weight=w for each of them. The weights and normalization factors are represented as such:

- avg_upload_ms (w=1.0, normalized by 138.42)
- p95_upload_ms (w=2.0, normalized by 208.37)
- avg_download_ms (w=1.0, normalized by 25.87)

- p95_download_ms (w=2.0, normalized by 28.33)
- reconstruction_time_ms (w=1.5, normalized by 37.83; only counted when failure_state=1)
- storage_overhead (w=2.0, normalized by 2.0; computed deterministically from EC-k-m as (k+m)/k)

$$\begin{aligned}\text{Score} = {} & \text{w}_1 \frac{\text{avg\_upload\_ms}}{M_{\text{avg,up}}} + \text{w}_2 \frac{\text{p95\_upload\_ms}}{M_{\text{p95,up}}} \\ & + \text{w}_3 \frac{\text{avg\_download\_ms}}{M_{\text{avg,down}}} + \text{w}_4 \frac{\text{p95\_download\_ms}}{M_{\text{p95,down}}} \\ & + \text{w}_5 \frac{\text{reconstruction\_time\_ms}}{M_{\text{recon}}} + \text{w}_6 \frac{\text{storage\_overhead}}{M_{\text{overhead}}}\end{aligned} \tag{1}$$

where:

- w_i are user-defined weights controlling the relative importance of each term,
- M_i are normalization constants (medians of the corresponding metrics computed on the test set),
- reconstruction_time_ms is included only when failure_state = 1,
- storage_overhead is computed deterministically from the EC parameters as (k+m)/k,
- The score is dimensionless, and lower values indicate better policies.

5.4. Classification-Based Design Specification

The proposed design uses a classifier-only policy recommendation method, where the machine learning model predicts the best EC policy directly. For each workload context, each policy run has 5 measured targets, which are: average upload time, average download time, p95 upload time, p95 download time, and reconstruction time. To define policy quality, a cost score is computed for each context:

$$S_{\text{total}} = W_{\text{perf}} \cdot S_{\text{perf,norm}} + W_{\text{storage}} \cdot S_{\text{storage}} \tag{2}$$

Where:

- $S_{\text{perf,norm}}$ is performance cost normalized within each context across policies.
- S_{storage} is the storage overhead penalty scaled by workload volume.
- $W_{\text{perf}}, W_{\text{storage}}$ are fixed manually.

Performance cost uses read/write weighting and reliability for p95:

$$\begin{aligned}S_{\text{perf}} = {} & \text{w}(\text{avg\_upload} + 2\rho\, \text{p95\_upload}) \\ & + \text{r}(\text{avg\_download} + 2\rho\, \text{p95\_download}) + 1.5\, \text{recon}\end{aligned} \tag{3}$$

Where:

- r = read_ratio, w = 1-r
- Reliability factor for p95:

$$\rho = \min\left(1,\ \frac{\text{num\_objects}}{T}\right) \tag{4}$$

Here, T = Threshold

The weights used for score calculation are determined based on metric importance:

$$W_{\text{perf}} = 0.65, \quad W_{\text{storage}} = 0.35$$

For each workload_id, performance score is normalized within the range of 0 and 1:

$$S_{\text{perf,norm}}(c, p) = \frac{S_{\text{perf}} - \min_p S_{\text{perf}}}{\max_p S_{\text{perf}} - \min_p S_{\text{perf}} + \varepsilon} \tag{5}$$

Storage cost is derived from EC overhead and normalized across candidate policies, then multiplied by normalized workload volume:

$$OH = \frac{\text{k} + \text{m}}{\text{k}} \tag{6}$$

Then we normalize OH across candidate policies:

$$OH_{\text{norm}} = \frac{OH - \min(OH)}{\max(OH) - \min(OH)} \tag{7}$$

Then we compute workload volume proxy:

$$V = \log\left(1 + \text{write\_mb}\right), \qquad V_{\text{norm}} = \frac{V - \min(V)}{\max(V) - \min(V)} \tag{8}$$

Finally storage penalty is computed for all workload contexts:

$$S_{\text{storage}} = OH_{\text{norm}} \cdot V_{\text{norm}} \tag{9}$$

We used three classifiers to train and evaluate the models, which are CatBoostClassifier, XGBoostClassifier, and Logistic Regression.

5.5. Data Preprocessing

To ensure consistency, valid learning, and prevent leakage, several columns are dropped- such as run_id, timestamp, which are identifiers. ec_k, ec_m, storage_overhead, and efficiency are dropped as redundant, as these can be derived from policy_name. Columns such as cpu_util_pct_during_run, data_disks_used_pct_after, and data_disks_used_pct_delta are dropped to avoid leaky signals. The missing values in time_since_last_failure are

ignored as CatBoost can handle NaN values.
For CatBoost & XGBoost, log target transformation was used in training (log1p target transformation) to reduce the effect of extreme values and ensure stabilization in training. The model learns

$$y' = \log(1 + y) \tag{10}$$

and predictions are inverse-transformed back into milliseconds as:

$$y = \exp(y') - 1 \tag{11}$$

For classifier-only implementation, a new categorical feature was introduced for object-count binning. This helps the model in capturing shifts between very small and very large workloads.

Data reduction is done based on two approaches:

- Feature level reduction: Keeping only pre-run, decision-relevant data and excluding constant values
- Reconstruction time: For reconstruction time, only rows with failure state=1 are considered to avoid polluting the learning

The dataset, after preprocessing, contains 30,000 rows, including 12,000 failure rows (40%). From the initial 53 extracted features, 29 relevant features were selected through preprocessing and feature selection procedures, while the target set consists of 5 columns.

6. Experimental Evaluation

In this chapter, we evaluate the effectiveness of the proposed ML-based EC policy recommendation framework. The experiments analyze how accurately models can recommend optimal EC policy under various workload contexts.

6.1. Experimental Setup

For training the ML models, we used Google Colab and Python libraries like numpy [34], sklearn [35], and pandas [36].

6.2. Performance Evaluation

Since the final decision is based on policy ranking, we compare and evaluate our regression and classification models based on top-k accuracy and regret statistics analysis to check recommendation reliability. We compare Top-k accuracy for each model, where Top-1 accuracy is the fraction of workload contexts where the model's highest-ranked policy matches the true best policy, while Top-3 accuracy is the fraction of contexts where the true best policy appears anywhere among the model's three highest-ranked policies. We also evaluate regret value to see the score gap between the chosen policy and the true best policy for the same context. Mean/median/p95 regret values summarize the tail behaviors.

Model	Top-1 (%)	Top-3 (%)	Mean regret	Median regret	P95 regret
CatBoost	47.17	87.17	1.46	0.063	10.23
XGBoost	48.16	79.66	1.806	0.089	11.557
Random Forest	41.97	100.00	0.0101	0.0021	0.0456

Table 2: Recommendation quality of regression-based models using Top-k accuracy and regret statistics.

From Table 2 in the regression-based recommendation design, XGBoost achieved the highest top-1 accuracy at 48.16%, followed by CatBoost at 47.17% and Random Forest at 41.97%. In top-3 accuracy, Random Forest Regression got the best performance at 100%, followed by CatBoost at 87.17%, with XGBoost at 79.66%. In regret metrics, Random Forest achieved the lowest mean (0.010139), median (0.002167), and p95 regret (0.045648), indicating a stable recommendation performance. CatBoost has moderate regret performance with a mean regret of 1.46, a median regret of 0.063, and a p95 regret of 10.23. XGBoost recorded the highest regret values with a mean regret of 1.806, a median regret of 0.089, and a p95 regret of 11.557.

From the regret CDF, we can see the cumulative fraction of workload contexts with regret less than or equal to that value.

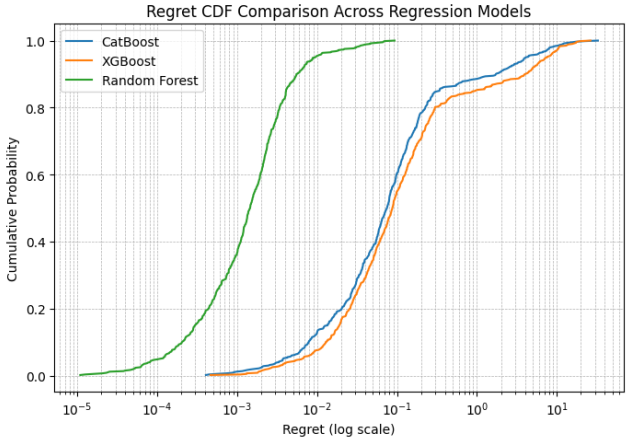

Figure 9: CDF of regression-based models

Here in Figure 9, the x-axis represents regret and the y-axis represents the total percentage of workload contexts. From the regression-based approach, we see that Random Forest rises earlier, meaning a larger fraction of cases have very low regret, while CatBoost and XGBoost rise later and show heavier tails.

The classification baseline/paradigm/approach also uses the same workload context to directly predict the best policies. The following table contains the top-k and regret values for the classification models.

Model	Top-1 (%)	Top-3 (%)	Mean regret	Median regret	P95 regret
CatBoostCls	19.56	48.44	0.1786	0.0946	0.6452
XGBoostCls	30.66	72.22	0.1137	0.0404	0.6432
Logistic Regression	19.56	46.67	0.1807	0.1026	0.6432

Table 3: Recommendation quality of classification-based models using Top-k accuracy and regret statistics.

From Table 3, we can see that all 3 models show comparatively weaker results than regressor models across the policy recommendation metrics here. Here we can see that XGBoostClassifier achieved the highest top-k accuracy, for which top-1 is 30.66% and top-3 is 72.22%. XGBoostClassifier also achieved the lowest mean regret, median regret, and p95 regret, which shows XGBoostClassifier performs overall better compared to CatBoostClassifier and Logistic Regression.

The regret CDF in Figure-6 shows the cumulative fraction of workload contexts for the classification models.

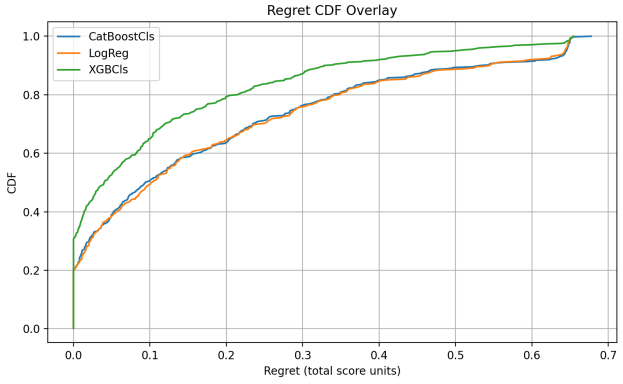

Figure 10: CDF of classification-based models

Here, the x-axis represents regret, and the y-axis represents the cumulative percentage of workload contexts with regret less than or equal to that value. Here, XGBoostClassifier rises earlier than CatBoostClassifier and Logistic Regression, meaning it produces low-regret decisions for a larger fraction of workload contexts. Also, CatBoostClassifier and Logistic Regression curves indicate higher regret across heavy-tailed cases.

6.3. Statistical Analysis

For regression model, CatBoost, a median regret of 0.063 means that at least half of all workloads get an optimal policy with bounded mean regret, confirming the limited performance loss even when the optimal policy is not selected. The CDF graph shows minimal probability of large deviation from the optimal path while proving a consistent choice of avoiding highly suboptimal policy selections.

For XGBoost regression model, the median regret of 0.089 and the mean regret of 1.806 mean that very little performance is lost even when the optimal policy is missed. The p95 regret of 11.557 further suggests that high-loss events occur only within a small fraction of workload and user contexts, proving a stable performance under different varying operational conditions. Random Forest Regression demonstrates the most stable policy ranking across multiple performance metrics. CDF showing performance remaining the most optimal for most recommendations. It also suggests low regret across environments due to consistent regression performance.

In contrast, classification-based approaches exhibit a more moderate median regret but substantially higher mean regret, indicating a right-skewed distribution where a limited number of contexts costs significant performance penalties. Elevated p95 regret further indicates susceptibility to high additional costs relative to oracle policy selection, reducing their reliability for automated EC policy recommendation.

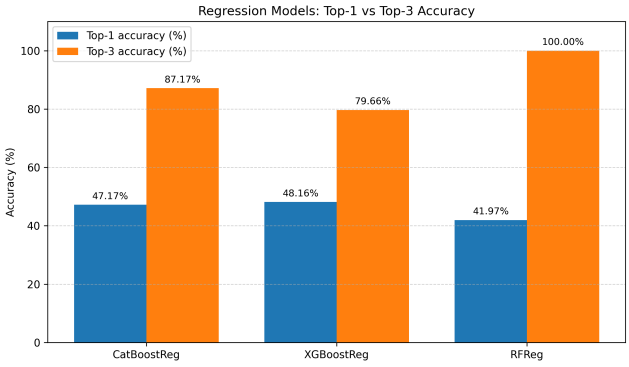

Figure 11: Top-K accuracy comparison for regression models

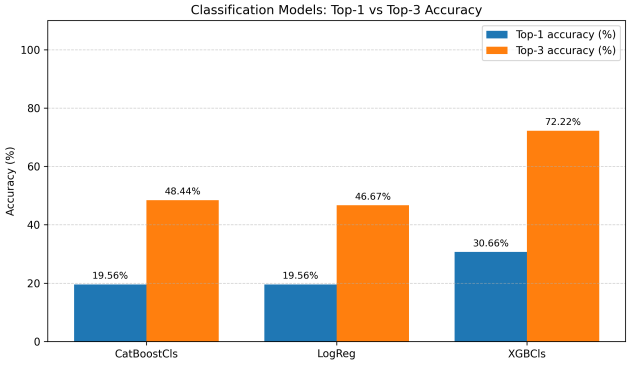

Figure 12: Top-K accuracy comparison for regression models

In the regression framework, CatBoost achieves a top-1 accuracy of 47.17% and a top-3 accuracy of 87.17%, demonstrating strong capability in capturing relative policy ranking despite ambiguity in precise ordering. Similarly, XGBoost achieves 48.16% top-1 accuracy and 79.66% top-3 accuracy across 600 test contexts, confirming the consistent ranking reliability. Random Forest Regression then further supports top-k recommendation by ranking policies according to weighted cost-based evaluation metrics.

Unlike regression, classification policies do not perform well. Within the classification paradigm, CatBoostClassifier and Logistic Regression achieve a low identical top-1 accuracy of 19.56%, while XGBoostClassifier achieves the highest among them with a top-1 accuracy of 30.66%. For top-3 accuracy, CatBoostClassifier and Logistic Regression achieve 48.44% and 46.67%, respectively, whereas XGBoostClassifier achieves 72.22%. These findings indicate that classification-based models demonstrate lower reliability in identifying optimal EC policies compared to regression-driven ranking methodologies.

6.4. Challenges and Limitations

The limitations of the regression models include the following:

- Because of the heavy-tailed nature of the dataset and the existence of extreme outliers, exact regression is a difficult task.
- The regret histogram and CDF show that a small number of workloads are showing predictions that show values different from the true best policy by a large margin, which enforces the need for top-3 instead of using only top-1.

In the case of classifier-only design, there are several reasons why all 3 models show poor results:

- Latency and reconstruction measurements can vary across different rows. Therefore, two policies may have identical scores for the same context. Small amount of noise can change the 'best policy' label in such situations.
- Oracle selection favours certain policies while some policies rarely appears. This affects the model's ability to learn minority-class patterns.
- Although reliability scaling helps to reduce noise, tail latencies (p95) remain difficult to use as stable learning signals, especially for low object counts.

6.5. Discussion

From our experiment, we can see the behavior of different model approaches by observing the results. It gives us an idea about how machine learning can relate to the characteristics of erasure-coded object storage systems. The results show the feasibility of Machine Learning to predict an optimal EC policy in workload variations. Here, the regression-based paradigm outperformed the classification approach. It successfully captures relative performance relationships among candidate policies, which is essential for workload-aware policy ranking. Furthermore, the high top-k accuracy achieved by the regression models indicates that, in most workload scenarios, the recommended policies remain close to the best-performing EC policy. The low regret values also indicate a very small margin of error, confirming the reliability of the regression-based recommendation framework. In contrast, the classification models are comparatively weak because of label instability and class imbalance. This is happening because multiple EC policies produce similar performance outcomes under the same workload context. Additionally, classification models treat policy selection as a fixed categorical decision, which limits their ability to capture subtle performance differences and ranking relationships among multiple candidate policies. Therefore, the proposed framework demonstrates practical applicability and deployability within controlled environments. Further system-level integration is required before it can be considered fully dependable across diverse real-world operating conditions.

7. Conclusion and Future Work

In this research, we investigated whether OpenStack Swift can be enhanced to become workload-aware and user-data-aware through a machine learning framework capable of adapting to dynamic workload and failure conditions. Due to the absence of publicly available datasets supporting workload-aware EC policy selection, we developed the ABDS-30k dataset as a core contribution of this work. The ABDS-30k dataset was created in a controlled experiment with 3,000 workload variations in 10 EC policies, capturing key performance metrics such as latency, reconstruction time, and recovery time in 30,000 situations under both normal and failure scenarios within a Swift All-In-One (SAIO) environment.

Using this dataset, the research evaluated two policy recommendation approaches: (i) regression-based and (ii) classification-based prediction. The regression-based approach demonstrated superior reliability, providing more consistent and efficient policy recommendations even when latency predictions were not perfectly accurate and helpful. The top-1 recommendation accuracy ranged between 42% and 56%, with the top-3 accuracy consistently exceeding 79%. It means the recommended policies were generally close to the optimal selection policy, with large performance losses occurring only in a small number of extreme workload contexts. In contrast, classification-based models showed lower reliability due to label instability caused by multiple EC policies exhibiting similar performance, sensitivity to noise, and class imbalance. Overall, the findings demonstrate that ranking EC policies based on predicted performance metrics is significantly more robust and dependable than directly predicting a single optimal policy in noisy and tail-heavy system environments.

Key contributions of this study are: (1) a real-system empirical EC dataset linking workload/failure context to policy behavior; (2) evidence that ML can learn meaningful patterns in erasure-coded storage despite heavy-tailed and noisy measurements; (3) a recommendation evaluation methodology emphasizing regret and top-k reliability (not only point prediction error); and (4) a practical foundation for future adaptive EC policy control-plane mechanisms

beyond static configuration.

Future work includes enabling online/adaptive learning beyond offline training; deeper integration with Swift telemetry/monitoring for stronger policy discrimination and stability; validation on multi-node production-scale clusters under realistic contention and distributed recovery; and testing cross-platform generalization to other object stores such as Ceph or MinIO.

References

[1] M. Liu, L. Pan, S. Liu, Cost optimization for cloud storage from user perspectives: Recent advances, taxonomy, and survey, ACM Comput. Surv. 55 (2023).

[2] J. Noor, R. I. Upoma, M. S. I. Sakif, A. A. A. Islam, Towards benchmarking erasure coding schemes in object storage system: A systematic review, Future Generation Computer Systems 163 (2025) 107522.

[3] J.-J. Kim, Erasure-coding-based storage and recovery for distributed exascale storage systems, Applied Sciences 11 (2021).

[4] J. C. W. Chan, Q. Ding, P. P. C. Lee, H. H. W. Chan, Parity logging with reserved space: towards efficient updates and recovery in erasure-coded clustered storage, in: Proceedings of the 12th USENIX Conference on File and Storage Technologies, FAST'14, USENIX Association, USA, 2014, p. 163–176.

[5] Griffiths, The latest cloud computing statistics (updated january 2025) | aag it support, 2025. URL: https://aag-it.com/the-latest-cloud-computing-statistics/.

[6] 2024. URL: https://www.gartner.com/en/newsroom/press-releases/2024-11-19-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-total-723-billion-dollars-in-2025?

[7] 2013. URL: https://ceph.io/en/news/blog/2013/benchmarking-ceph-erasure-code-plugins/.

[8] ???? URL: https://docs.openstack.org/swift/latest/overview_policies.html.

[9] D. Durner, V. Leis, T. Neumann, Exploiting cloud object storage for high-performance analytics, Proceedings of the VLDB Endowment 16 (2023) 2769–2782.

[10] C. Huang, H. Simitci, Y. Xu, A. Ogus, B. Calder, P. Gopalan, J. Li, S. Yekhanin, Erasure coding in windows azure storage, in: Proceedings of the 2012 USENIX Conference on Annual Technical Conference, USENIX ATC'12, USENIX Association, USA, 2012, p. 2.

[11] X. Pei, Y. Wang, X. Ma, F. Xu, Efficient in-place update with grouped and pipelined data transmission in erasure-coded storage systems, Future Generation Computer Systems 69 (2016) 24–40.

[12] J. Hu, J. Kosaian, K. V. Rashmi, Rethinking erasure-coding libraries in the age of optimized machine learning, in: Proceedings of the 16th ACM Workshop on Hot Topics in Storage and File Systems, HotStorage '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 23–30. URL: https://doi.org/10.1145/3655038.3665943. doi:10.1145/3655038.3665943.

[13] M. Cui, Y. Wang, Ec-kad: An efficient data redundancy scheme for cloud storage, Electronics 14 (2025).

[14] F. Xu, Y. Wang, X. Ma, Incremental encoding for erasure-coded cross-datacenters cloud storage, Future Generation Computer Systems 87 (2018) 527–537.

[15] Y. Chen, Q. Ke, H. Li, Y. Wu, Y. Zhang, x meta: Ssd-hdd-hybrid optimization for metadata maintenance of cloud-scale object storage, ACM Transactions on Architecture and Code Optimization 21 (2024) 1–20.

[16] V. Bucur, C. Dehelean, L. Miclea, Object storage in the cloud and multi-cloud: State of the art and the research challenges, in: 2018 IEEE International Conference on Automation, Quality and Testing, Robotics (AQTR), 2018, pp. 1–6. doi:10.1109/AQTR.2018.8402762.

[17] J. Matt, P. Waibel, S. Schulte, Cost- and latency-efficient redundant data storage in the cloud, in: 2017 IEEE 10th Conference on Service-Oriented Computing and Applications (SOCA), 2017, pp. 164–172. doi:10.1109/SOCA.2017.30.

[18] M. Factor, K. Meth, D. Naor, O. Rodeh, J. Satran, Object storage: the future building block for storage systems, in: 2005 IEEE International Symposium on Mass Storage Systems and Technology, 2005, pp. 119–123. doi:10.1109/LGDI.2005.1612479.

[19] Y. Su, D. Feng, Y. Hua, Z. Shi, Predicting response latency percentiles for cloud object storage systems, in: 2017 46th International Conference on Parallel Processing (ICPP), 2017, pp. 241–250. doi:10.1109/ICPP.2017.33.

[20] M. Akbar, I. Ahmad, M. Mirza, M. Ali, P. Barmavatu, Enhanced authentication for de-duplication of big data on cloud storage system using machine learning approach, Cluster Computing 27 (2023) 3683–3702.

[21] G. Levitin, L. Xing, Y. Dai, Optimizing uploading and downloading pace distribution in system with two non-identical storage units, Reliability Engineering System Safety 231 (2022) 109017.

[22] M. Saadoon, S. H. A. Hamid, H. Sofian, H. H. Altarturi, Z. H. Azizul, N. Nasuha, Fault tolerance in big data storage and processing systems: A review on challenges and solutions, Ain Shams Engineering Journal 13 (2021) 101538.

[23] A. S. Mondal, A. Mukhopadhyay, S. Chattopadhyay, Machine learning-driven automatic storage space recommendation for object-based cloud storage system, Complex Intelligent Systems 8 (2021) 489–505.

[24] R. R. Noel, R. Mehra, P. Lama, Towards self-managing cloud storage with reinforcement learning, 2019 IEEE International Conference on Cloud Engineering (2019) 34–44.

[25] S. Syed, E. M. Albalawi, Optimizing cloud resource allocation with machine learning: A comprehensive approach to efficiency and performance, Research Square (Research Square) (2024).

[26] E. A. T. Kamble, Predictive resource allocation strategies for cloud computing environments using machine learning, Deleted Journal 19 (2024) 68–77.

[27] Managing machine learning lifecycle in oracle cloud infrastructure for erp-related use cases, International Journal of Emerging Research in Engineering and Technology 4 (2023).

[28] D. R. Patil, Dynamic resource allocation and memory management using machine learning for cloud environments, International Journal of Advanced Trends in Computer Science and Engineering 9 (2020) 5921–5927.

[29] Z. Su, Y. Wang, T. H. Luan, N. Zhang, F. Li, T. Chen, H. Cao, Secure and efficient federated learning for smart grid with edge-cloud collaboration, IEEE Transactions on Industrial Informatics 18 (2021) 1333–1344.

[30] Y. Wang, Q. Bao, J. Wang, G. Su, X. Xu, Cloud computing for large-scale resource computation and storage in machine learning, Journal of Theory and Practice of Engineering Science 4 (2024) 163–171.

[31] Abds-30k, ???? URL: https://docs.google.com/spreadsheets/d/1scBzznN_-DJH7jX8JjdNQ0ebZtpdS53GHeQmrBTATeo/edit?usp=sharing.

[32] Train series release notes — swift release notes documentation, ???? URL: https://docs.openstack.org/releasenotes/swift/train.html.

[33] Erasure code support, ???? URL: https://www.gartner.com/en/newsroom/press-releases/2024-11-19-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-total-723-billion-dollars-in-2025?

[34] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, Nature 585 (2020) 357–362.

[35] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine

Learning Research 12 (2011) 2825–2830.
[36] T. pandas development team, pandas-dev/pandas: Pandas, 2020. URL: https://doi.org/10.5281/zenodo.3509134. doi:10.5281/zenodo.3509134.